Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
УРАЛЬСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ
ИМЕНИ ПЕРВОГО ПРЕЗИДЕНТА РОССИИ Б.Н. ЕЛЬЦИНА
(УрФУ имени первого Президента России Б.Н. Ельцина)
Институт радиоэлектроники и информационных технологий — РТФ
Школа профессионального и академического образования

ВВЕДЕНИЕ В АЛГОРИТМЫ И СТРУКТУРЫ ДАННЫХ. ИССЛЕДОВАНИЕ ВОЗМОЖНОСТЕЙ КОЛЛЕКЦИЙ В ЯЗЫКЕ ПРОГРАММИРОВАНИЯ JAVA

Отчет по лабораторной работе №3

по дисциплине «Программирование на Java»

	Дата	Подпись	
Преподаватель:	_____	_____	Архипов Н.А.
Студенты:	_____	_____	Пантелеев Е.А.

Группа: РИМ-150950

Екатеринбург

2026

ЦЕЛЬ РАБОТЫ

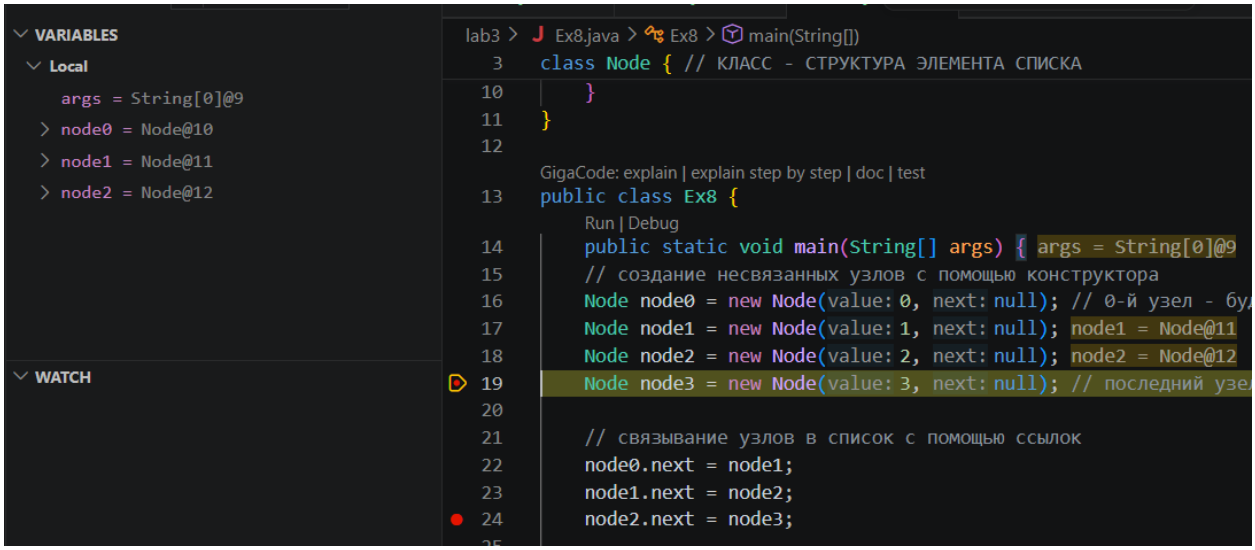
Приобретение навыков работы с рекурсивными методами, знакомство с динамическими структурами данных, приобретение навыков создания и использования простейшей динамической структуры, исследование возможностей коллекций в языке программирования Java, сравнение коллекций по времени выполнения операций, знакомство с реализацией коллекций.

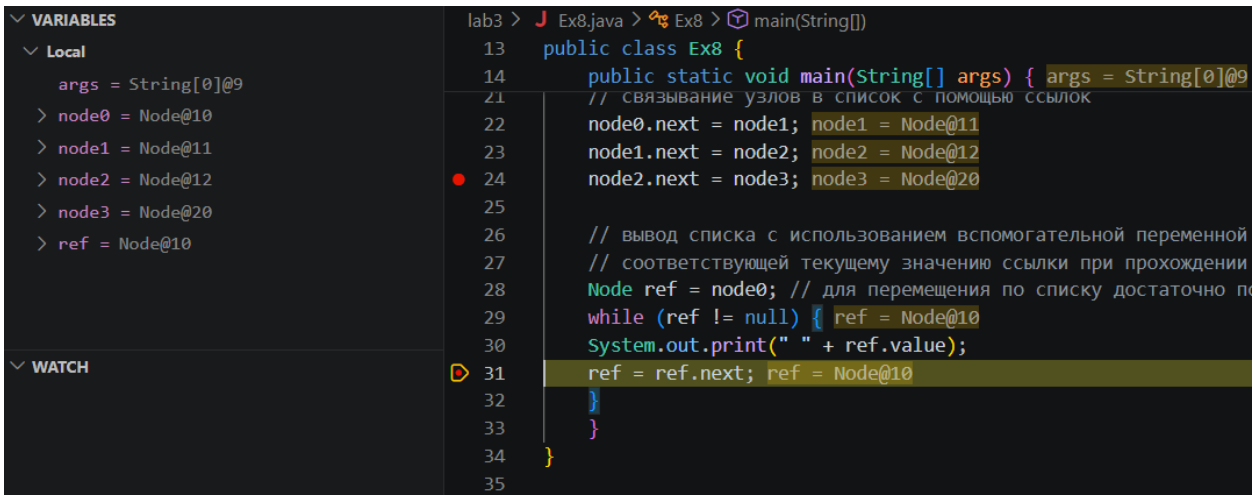
ХОД РАБОТЫ

1. Задания для самостоятельной работы

Для экономии места в файле, со всем кодом можно ознакомиться в репозитории. Здесь будут приведены только необходимые скриншоты.

1.4 Пример





```
VARIABLES
Local
  args = String[0]@9
  node0 = Node@10
  node1 = Node@11
  node2 = Node@12
  node3 = Node@20
  ref = Node@12
WATCH
```

```
lab3 > Ex8.java > Ex8 > main(String[])
13  public class Ex8 {
14      public static void main(String[] args) { args = String[0]@9
21      // связывание узлов в список с помощью ссылок
22      node0.next = node1; node1 = Node@11
23      node1.next = node2; node2 = Node@12
24      node2.next = node3; node3 = Node@20
25
26      // вывод списка с использованием вспомогательной переменной
27      // соответствующей текущему значению ссылки при прохождении
28      Node ref = node0; // для перемещения по списку достаточно по
29      while (ref != null) { ref = Node@12
30      System.out.print(" " + ref.value);
31      ref = ref.next; ref = Node@12
32      }
33      }
34  }
```

Пример 1.6

ArrayList быстрее LinkedList, так как его сложность обращения к элементу O(1), а у LinkedList – O(n).

Пример 1.7

```
VARIABLES
Local
  args = String[0]@9
  head = Node@16
  i = 8
WATCH
```

```
lab3 > Ex111.java > Ex111 > main(String[])
1  package lab3;
2
   GigaCode: explain | explain step by step | doc | test
3  public class Ex111 {
       Run | Debug
4      public static void main(String[] args) { args = String[0]@
5          // наращивание с головы
6          Node head = null; head = Node@16
7          for (int i = 9; i >= 0; i--) { i = 8
8              head = new Node(i, head); head = Node@16, i = 8
9          }
10
11         // вывод
12         Node ref = head;
13         while (ref != null) {
```

```
VARIABLES
Local
  args = String[0]@9
  head = Node@20
  i = 5
WATCH
```

```
lab3 > Ex111.java > Ex111 > main(String[])
1  package lab3;
2
   GigaCode: explain | explain step by step | doc | test
3  public class Ex111 {
       Run | Debug
4      public static void main(String[] args) { args = String[0]@9
5          // наращивание с головы
6          Node head = null; head = Node@20
7          for (int i = 9; i >= 0; i--) { i = 5
8              head = new Node(i, head); head = Node@20, i = 5
9          }
10
11         // вывод
```

```
VARIABLES
Local
  args = String[0]@9
  head = Node@26
  ref = Node@25
WATCH
CALL STACK
Thread [main]  PAUSED ON BREAKPOINT
```

lab3 > Ex111.java > Ex111 > main(String[])

```
1   package lab3;
2
    GigaCode: explain | explain step by step | doc | test
3   public class Ex111 {
        Run | Debug
4       public static void main(String[] args) { args = String[0]@9
5           // наращивание с головы
6           Node head = null; head = Node@26
7           for (int i = 9; i >= 0; i--) {
8               head = new Node(i, head);
9           }
10
11          // вывод
12          Node ref = head; ref = Node@25, head = Node@26
13          while (ref != null) { ref = Node@25
14              System.out.print(" " + ref.value);
15              ref = ref.next; ref = Node@25
16          }
17      }
```

```
VARIABLES
Local
  args = String[0]@9
  head = Node@10
  tail = Node@16
  i = 4
  newNode = Node@17
WATCH
CALL STACK
Thread [main]  PAUSED ON BREAKPOINT
```

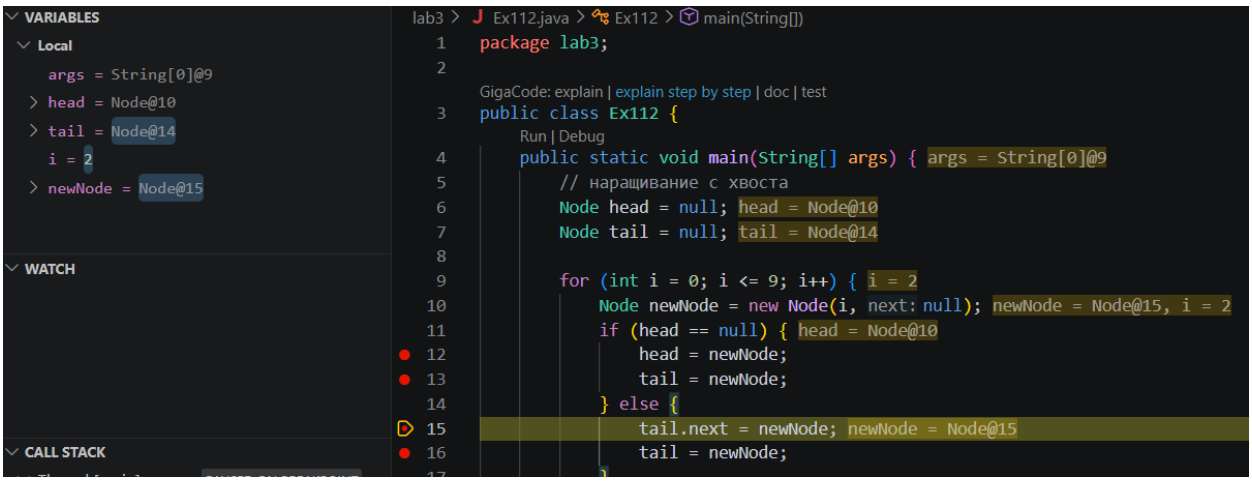
lab3 > Ex112.java > Ex112 > main(String[])

```
1   package lab3;
2
    GigaCode: explain | explain step by step | doc | test
3   public class Ex112 {
        Run | Debug
4       public static void main(String[] args) { args = String[0]@9
5           // наращивание с хвоста
6           Node head = null; head = Node@10
7           Node tail = null; tail = Node@16
8
9           for (int i = 0; i <= 9; i++) { i = 4
10              Node newNode = new Node(i, next: null); newNode = Node@17, i = 4
11              if (head == null) { head = Node@10
12                  head = newNode;
13                  tail = newNode;
14              } else {
15                  tail.next = newNode; newNode = Node@17
16                  tail = newNode;
17              }
18          }
```

```
VARIABLES
Local
  args = String[0]@9
  head = Node@10
  tail = Node@17
  i = 5
  newNode = Node@18
WATCH
CALL STACK
```

lab3 > Ex112.java > Ex112 > main(String[])

```
1   package lab3;
2
    GigaCode: explain | explain step by step | doc | test
3   public class Ex112 {
        Run | Debug
4       public static void main(String[] args) { args = String[0]@9
5           // наращивание с хвоста
6           Node head = null; head = Node@10
7           Node tail = null; tail = Node@17
8
9           for (int i = 0; i <= 9; i++) { i = 5
10              Node newNode = new Node(i, next: null); newNode = Node@18, i = 5
11              if (head == null) { head = Node@10
12                  head = newNode;
13                  tail = newNode;
14              } else {
15                  tail.next = newNode; newNode = Node@18
16                  tail = newNode;
```



2. Общее описание коллекций в java

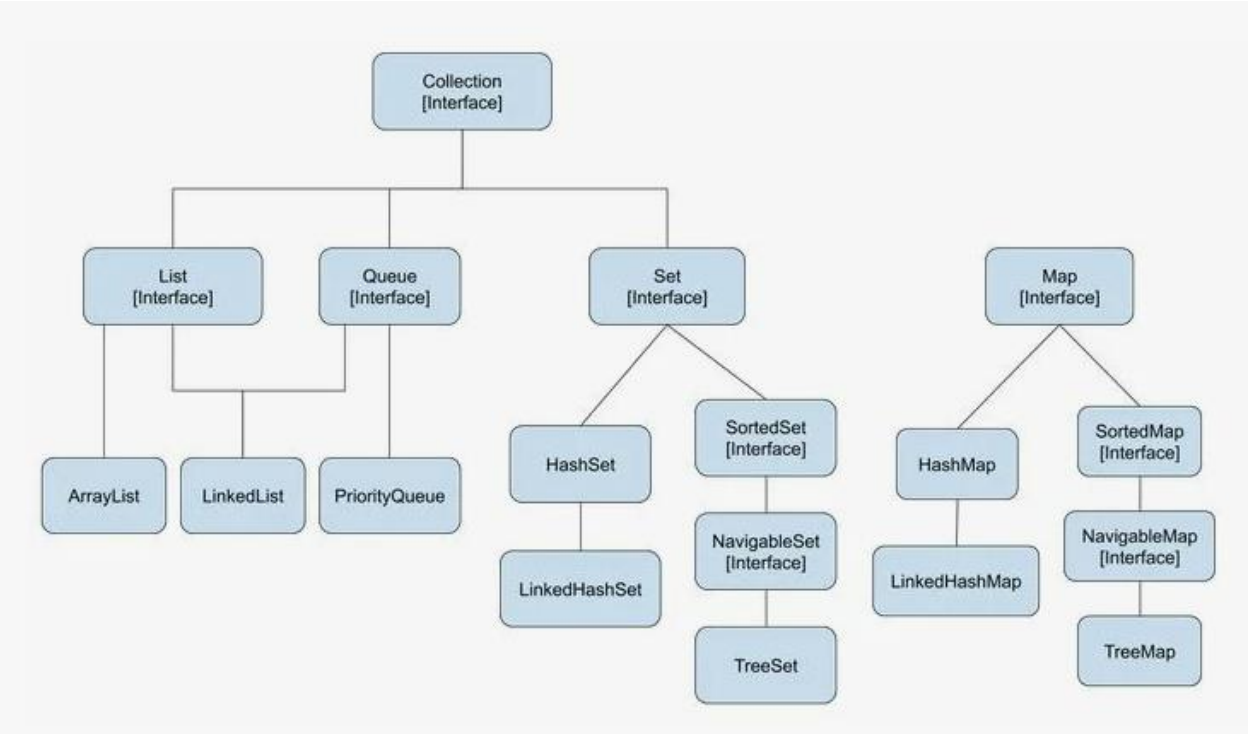
Collection – корневой интерфейс иерархии коллекций в Java. Он определяет основные операции для работы с группами объектов (списки, множества, очереди).

Метод	**Описание**
boolean add(E e)	Добавляет элемент в коллекцию
boolean remove(Object o)	Удаляет указанный элемент
boolean contains(Object o)	Проверяет наличие элемента
int size()	Возвращает количество элементов
boolean isEmpty()	Проверяет, пуста ли коллекция
void clear()	Удаляет все элементы
Iterator<E> iterator()	Возвращает итератор для обхода
Object[] toArray()	Преобразует коллекцию в массив

Map – интерфейс для структур данных, хранящих пары "ключ-значение". Каждый ключ уникален и связан с одним значением.

Метод	**Описание**
V put(K key, V value)	Добавляет пару ключ-значение
V get(Object key)	Возвращает значение по ключу
V remove(Object key)	Удаляет пару по ключу
boolean containsKey(Object key)	Проверяет наличие ключа
boolean containsValue(Object value)	Проверяет наличие значения

Set<K> keySet()	Возвращает множество ключей
Collection<V> values()	Возвращает коллекцию значений
Set<Map.Entry<K,V>> entrySet()	Возвращает множество пар



3. Описание коллекций (в соответствии с вариантом)

ArrayList

- Элементы хранятся в обычном массиве (Object[]) внутри класса
- Упорядоченно - порядок добавления сохраняется
- Элементы хранятся последовательно в памяти (смежные ячейки)

Операция	**Вычислительная сложность**	**Как работает**
Добавление в конец	O(1) амортизированно	Проверяет, есть ли место; если нет — создает новый массив в 1.5 раза больше и копирует все элементы
Добавление в середину	O(n)	Сдвигает все элементы правее позиции вставки
Удаление по индексу	O(n)	Сдвигает все элементы левее позиции удаления

Получение по индексу	O(1)	Прямой доступ к элементу массива по индексу
Поиск по значению	O(n)	Линейный перебор до нахождения элемента

TreeMap

- Хранится в красно-черном дереве (сбалансированное двоичное дерево поиска)
- Упорядоченно по ключам (естественный порядок или Comparator)
- Каждый узел содержит: ключ, значение, ссылки на левого/правого потомка, цвет (красный/черный)

Операция	**Вычислительная сложность**	**Как работает**
Добавление	O(log n)	Вставка узла в дерево с последующей балансировкой (вращения, перекрашивание)
Удаление	O(log n)	Поиск узла, удаление с последующей балансировкой дерева
Получение по ключу	O(log n)	Двоичный поиск: сравнение ключа, спуск влево/вправо
Поиск по значению	O(n)	Полный обход дерева

ArrayDeque

- Хранится в кольцевом массиве (circular array)
- Упорядоченно – элементы в порядке добавления
- Два указателя: head (начало) и tail (конец)

Операция	**Вычислительная сложность**	**Как работает**

Добавление в начало	O(1)	head-- (с учетом кольцевой структуры)
Добавление в конец	O(1)	tail++ (с учетом кольцевой структуры)
Удаление из начала	O(1)	head++ (с учетом кольцевой структуры)
Удаление из конца	O(1)	tail-- (с учетом кольцевой структуры)
Получение по индексу	O(1)	Вычисление реальной позиции: (head + index) % capacity
Поиск по значению	O(n)	Линейный перебор

4. Сравнение коллекций (в соответствии с вариантом)

Вариант 12. Коллекции ArrayList, TreeMap, ArrayDeque. В TreeMap все хранится в ключ-значение, поэтому понятие начало, середина и конец коллекции там нет.Также в связи со значительной нагрузкой на компьютер размер выборок был уменьшен в 10 раз.

Таблица 1. Сравнение времени выполнения операции добавления

Коллекция	в начало коллекции	в середину коллекции	в конец коллекции
ArrayList	3691 ms	745 ms	29 ms
TreeMap	-	-	-
ArrayDeque	7 ms	292142 ms	88 ms

Таблица 2. Сравнение времени выполнения операции удаления

Коллекция	в начале коллекции	в середине коллекции	в конце коллекции
ArrayList	5506 ms	1100 ms	1 ms

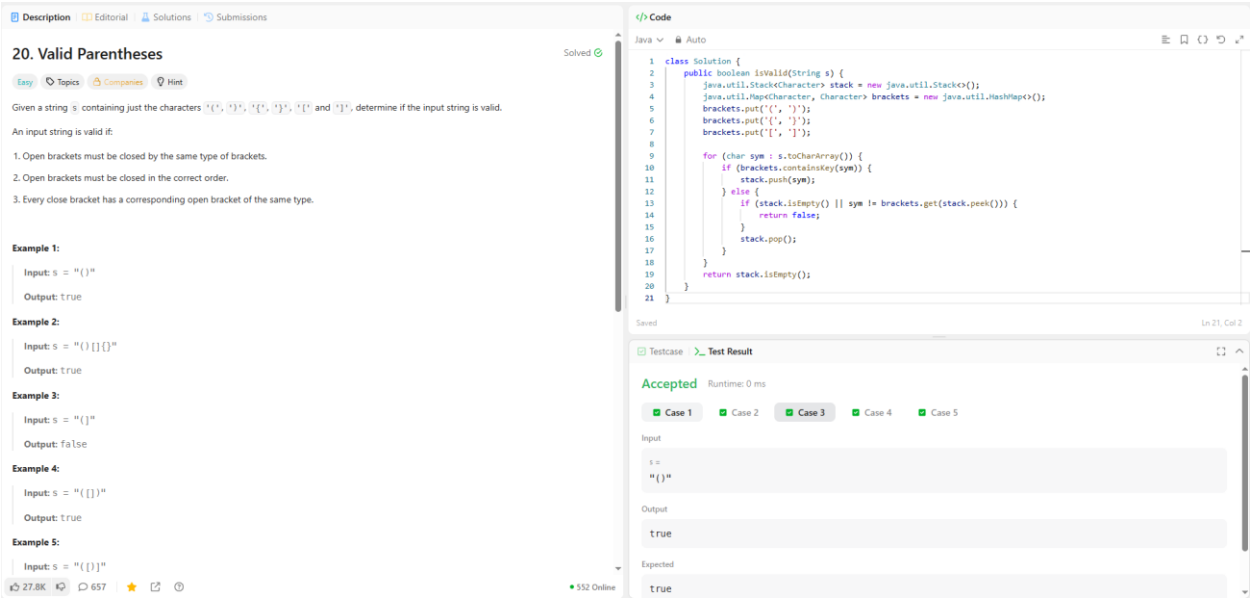
TreeMap	-	-	-
ArrayDeque	0 ms	103813 ms	1 ms

Таблица 3. Сравнение времени выполнения операции получения элемента по индексу

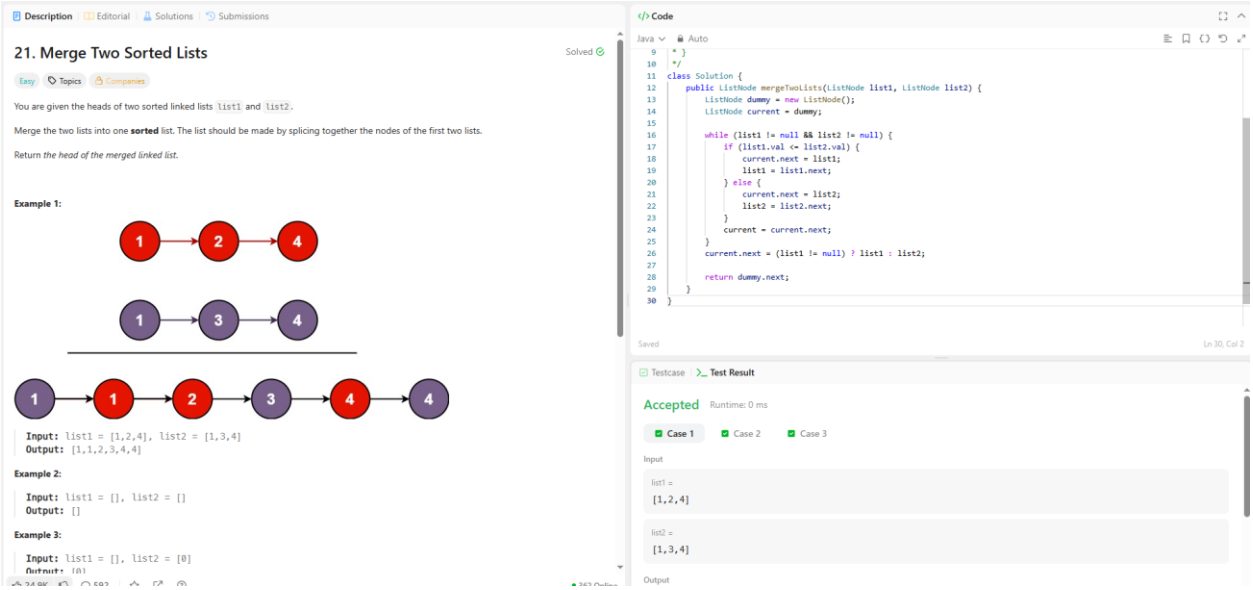
Коллекция	По индексу для <n> элементов
ArrayList	9368 ms
TreeMap	-
ArrayDeque	-

5. Задачи Leetcode

Задача 20. Valid parentheses



Задача 21. Merge two sorted lists



6. Ответы на вопросы

1. В чем разница между fail-safe и fail-fast свойствами?

fail-fast - выбрасывают ConcurrentModificationException при попытке модификации коллекции во время итерации (например, ArrayList). fail-safe - работают с копией данных, не выбрасывают исключение (например, ConcurrentHashMap).

2. В чем разница между Iterator и Enumeration?

Enumeration - устаревший интерфейс, не поддерживает удаление элементов; Iterator - современный, поддерживает remove().

3. В чем разница между Iterator и ListIterator?

ListIterator расширяет Iterator, работает только с List, позволяет проход в обоих направлениях (hasPrevious()), добавление и изменение элементов (add(), set()).

4. Дайте определение понятию “итератор”.

Итератор - объект, реализующий паттерн для последовательного обхода элементов коллекции без раскрытия её внутренней структуры.

5. Как задается порядок следования объектов в коллекции, как отсортировать коллекцию?

Порядок задается через Comparable (естественный порядок) или Comparator. Сортировка: Collections.sort(list) или list.sort(comparator).

6. Как получить коллекцию только для чтения?

Collections.unmodifiableList(), unmodifiableSet(), unmodifiableMap() и т.д. Возвращают view коллекции, которая выбрасывает UnsupportedOperationException при попытке изменения.

7. Как получить не модифицируемую коллекцию?

Collections.unmodifiableList(), unmodifiableSet(), unmodifiableMap() и т.д. Возвращают view коллекции, которая выбрасывает UnsupportedOperationException при попытке изменения.

8. Как получить синхронизированную коллекцию из не синхронизированной?

Collections.synchronizedList(), synchronizedSet(), synchronizedMap() - оборачивают коллекцию, синхронизируя все методы.

9. Как реализован цикл foreach?

Цикл for (T item : collection) - синтаксический сахар, компилируется в for (Iterator<T> it = collection.iterator(); it.hasNext();) с вызовом it.next().

10. Какие есть способы перебора всех элементов List?

Обычный for по индексу, foreach, Iterator, ListIterator, forEach() (лямбда), стримы (stream().forEach()).

11. Какие коллекции синхронизированы?

Vector, Stack, Hashtable (устаревшие); Collections.synchronized* обертки; ConcurrentHashMap, CopyOnWriteArrayList - потокобезопасны, но не синхронизированы через synchronized.

12. Какую функциональность представляет класс Collections?

Collections - утилитный класс с методами для сортировки, поиска, синхронизации, создания неизменяемых коллекций, заполнения, переворота и т.д.

13. Почему Map не наследуется от Collection?

Map - другая иерархия (ключ-значение), не подходит под контракт Collection (который работает с одиночными элементами).

14. Почему в классе iterator нет метода для получения следующего элемента без передвижения курсора?

Нарушило бы простоту итератора: увеличение сложности реализации, необходимость хранения состояния для "заглядывания вперед".

15. Почему нет метода iterator.add() чтобы добавить элементы в коллекцию?

Добавление через итератор неоднозначно: зависит от позиции вставки, не для всех коллекций имеет смысл. В ListIterator он есть.

16. Расскажите о реализации классов HashSet и TreeSet?

HashSet - на основе HashMap, неупорядочен, константное время. TreeSet - на основе TreeMap (красно-черное дерево), отсортирован, O(log n).

17. Чем отличаются ArrayDequeи TreeMap? Как они устроены и работают? Что со временем доступа к объектам, какие зависимости?

ArrayDeque - массив с двумя указателями (голова/хвост), O(1) для операций на концах. TreeMap – красно-черное дерево, O(log n) для вставки/поиска. Разное назначение: дек vs отсортированная мапа.

18. Что будет, если в Map положить два значения с одинаковым ключом?

Второе значение перезаписывает первое. Метод put() возвращает старое значение.

19. Что делать, чтобы не возникло исключение ConcurrentModificationException?

Использовать потокобезопасные коллекции (ConcurrentHashMap, CopyOnWriteArrayList) или синхронизировать итерацию внешним блоком, либо не модифицировать коллекцию во время обхода.

20. Что такое Hashtable, чем она отличается от HashMap? На сегодняшний день она deprecated, как все-таки использовать нужную функциональность?

Hashtable – устаревшая синхронизированная мапа, не разрешает null ключи/значения. Замена: ConcurrentHashMap для многопоточности, HashMap с Collections.synchronizedMap() при необходимости.

7. Ссылка на github

https://github.com/Fothiss/java_apps

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы были приобретены навыки работы с рекурсивными методами, знакомство с динамическими структурами данных, приобретение навыки создания и использования простейшей динамической структуры.